

Rapport de Projet IoT

N°4IASDR8

Sous le thème

Architecture IoT – Smart School

Système de Surveillance Intelligent
pour Établissements Scolaires

Établissement : École Marocaine des Sciences de l'Ingénieur

Filière : Data & ia

Réalisé par : Soufiane Kelmous et Abdellah Elmir

Encadrent : Mr Anass Deroussi

Année académique : 2025/2026

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce projet.

Mes remerciements s'adressent à notre encadrant, pour son suivi, sa disponibilité et ses remarques pertinentes qui nous ont permis de progresser tout au long de ce travail. Sa maîtrise des systèmes embarqués et de l'Internet des Objets nous a offert une vision claire et rigoureuse des enjeux technologiques actuels.

Je suis reconnaissant envers l'établissement EMSI, pour sa bienveillance, son accompagnement et les échanges constructifs qui ont largement contribué à l'enrichissement de nos compétences en développement IoT, en programmation embarquée et en architecture logicielle.

Résumé

Ce projet vise à développer un système intelligent capable de surveiller automatiquement l'environnement des salles de classe et de contrôler les équipements d'un établissement scolaire via une architecture IoT complète. Nous avons choisi d'exploiter les microcontrôleurs ESP32 grâce à une approche multi-couches, en intégrant plusieurs types de capteurs : DHT22 pour la température et l'humidité, capteur PIR pour la détection de mouvement, LDR pour la luminosité, capteur sonore, MQ-135 pour la qualité de l'air, capteur infrarouge pour la présence des élèves, et un module relais pour le contrôle des serrures électroniques.

Afin d'assurer une communication fiable et légère entre les nœuds IoT et le serveur, nous avons adopté le protocole MQTT (broker HiveMQ), complété par une API REST exposée par un serveur Node.js. Les données sont persistées dans une base MySQL et diffusées en temps réel vers un tableau de bord web via WebSocket (Socket.IO). L'ensemble de la solution a été validé par simulation sur la plateforme Wokwi, démontrant des latences inférieures à 1,5 seconde et un taux de livraison des messages MQTT supérieur à 99 %.

Ce travail ouvre des perspectives pour des applications avancées dans la gestion intelligente des établissements scolaires, l'optimisation énergétique et la sécurité des locaux.

Mots-clés : IoT, ESP32, Arduino, MQTT, Smart School, capteurs, tableau de bord, Wi-Fi, Node.js, MySQL, Wokwi, système embarqué.

Abstract

This project aims to develop an intelligent system capable of automatically monitoring the classroom environment and controlling the equipment of a school using a complete IoT architecture. We chose to leverage ESP32 microcontrollers through a multi-layer approach, integrating several types of sensors: DHT22 for temperature and humidity, PIR sensor for motion detection, LDR for luminosity, sound sensor, MQ-135 for air quality, infrared sensor for student presence detection, and a relay module for electronic door lock control.

To ensure reliable and lightweight communication between IoT nodes and the server, we adopted the MQTT protocol (HiveMQ broker), complemented by a REST API exposed by a Node.js server. Data is persisted in a MySQL database and broadcast in real time to a web dashboard via WebSocket (Socket.IO). The complete solution was validated through simulation on the Wokwi platform, demonstrating end-to-end latencies below 1.5 seconds and an MQTT message delivery rate above 99%.

This work opens avenues for advanced applications in smart school management, energy optimization, and premises security.

Keywords: IoT, ESP32, Arduino, MQTT, Smart School, sensors, dashboard, Wi-Fi, Node.js, MySQL, Wokwi, embedded systems.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Liste des Figures	vi
Introduction générale	1
1 Contexte du projet	2
1.1 Introduction	2
1.2 Présentation générale du projet	2
1.2.1 Problématique	2
1.2.2 Étude de l'existant	3
1.3 Objectifs du projet	3
1.4 Solution proposée	3
1.5 Conclusion	3
2 Étude des besoins	4
2.1 Introduction	4
2.2 Besoins fonctionnels	4
2.3 Besoins non-fonctionnels	5
2.4 Description des acteurs	5
2.5 Conclusion	5
3 Analyse et conception	6
3.1 Introduction	6
3.2 Outils de modélisation utilisés	6
3.3 Diagramme de cas d'utilisation	6
3.4 Diagrammes de séquence	7
3.5 Diagramme de classes	8
3.6 Conclusion	9
4 Étude technique	10
4.1 Introduction	10
4.2 Langages utilisés	10

4.3	Transfert d'architecture avec l'ESP32	10
4.4	Augmentation de données : rotation, zoom, recadrage	11
4.5	Prétraitement avec preprocess_input	12
4.6	Entraînement en deux phases	12
4.7	Évaluation sur le jeu de test	12
4.8	Sauvegarde du modèle final	13
4.9	Conclusion	13
5	Réalisation	14
5.1	Introduction	14
5.2	Page d'accueil	14
5.3	Tableau de bord principal	15
5.4	Carte d'une salle : état normal et verrouillage	16
5.5	Système d'alertes en temps réel	18
5.6	Test de propagation des seuils via le DHT22	19
5.7	Simulation du circuit ESP32 sur Wokwi	21
5.8	Conclusion	22
	Conclusion générale	24
	Références	25

Liste des Figures

3.1	Diagramme de cas d'utilisation du système IoT Smart School	7
3.2	Diagramme de séquence global du système Smart School IoT	8
3.3	Diagramme de classes : Structure et relations	9
4.1	Architecture technique complète du système	11
4.2	Évolution de la latence (simulation Wokwi)	13
5.1	Page d'accueil de l'application Smart School Dashboard	14
5.2	Vue d'ensemble du tableau de bord (KPI + 3 salles)	15
5.3	Carte de la Salle 1 -- mouvement détecté (badge MVT)	16
5.4	Carte de la Salle 1 -- après commande de verrouillage	17
5.5	Trois alertes simultanées sur la Salle 4 + journal MQTT	18
5.6	Manipulation du capteur DHT22 dans Wokwi	19
5.7	Propagation immédiate de la mesure côté dashboard	20
5.8	Alerte WARNING générée dans le journal	20
5.9	Simulateur Wokwi avec barre d'outils	21
5.10	Vue rapprochée du circuit (ESP32 + périphériques)	22
5.11	Manipulation du capteur PIR dans Wokwi	22

Introduction générale

La gestion intelligente des établissements scolaires est un domaine en pleine expansion, largement stimulé par les progrès de l'Internet des Objets (IoT). Les microcontrôleurs connectés, les capteurs intelligents et les plateformes cloud convergent désormais pour créer des environnements capables de percevoir, d'analyser et d'agir sur leur environnement physique en temps réel.

L'objectif de ce projet est de développer un système robuste et précis capable de surveiller automatiquement les conditions environnementales des salles de classe, de détecter la présence des élèves, de contrôler les accès via des serrures électroniques, et de notifier les responsables en cas d'anomalie. Pour ce faire, nous exploitons les microcontrôleurs ESP32, qui constituent une plateforme puissante et économique pour le développement IoT.

Nous adoptons une architecture IoT en quatre couches : couche capteurs/actionneurs, couche microcontrôleur, couche réseau (MQTT/HTTP) et couche applicative (Node.js + MySQL + Dashboard Web). Cette approche permet de réduire les exigences en termes d'infrastructure et d'accélérer le développement grâce à des protocoles et des bibliothèques open-source éprouvés.

Ce rapport présente le contexte, les besoins, l'analyse, la conception, l'étude technique et la réalisation du projet. Il se conclut par une évaluation des résultats et des perspectives d'amélioration.

Chapter 1

Contexte du projet

1.1 Introduction

La surveillance intelligente des établissements scolaires est un domaine en pleine expansion, largement stimulé par les progrès de l’IoT et plus particulièrement des systèmes embarqués connectés. Les microcontrôleurs ESP32 et les protocoles légers comme MQTT ont transformé la manière dont les bâtiments interagissent avec leur environnement, permettant des performances remarquables dans la collecte et l’exploitation des données physiques.

1.2 Présentation générale du projet

Ce projet consiste à concevoir un système automatique de surveillance et de contrôle des salles de classe. À partir d’un réseau de capteurs variés, l’objectif est de collecter les données environnementales, de détecter la présence des élèves et de permettre le contrôle à distance des équipements, en tenant compte des contraintes de latence, de sécurité et de consommation énergétique.

1.2.1 Problématique

Les principaux défis résident dans la grande diversité des données à collecter (capteurs analogiques et numériques), la nécessité d’une communication fiable et temps réel entre les nœuds IoT et le serveur, et la conception d’une interface utilisateur intuitive adaptée à différents profils (administrateur, enseignant, technicien). S’ajoutent à cela les contraintes de sécurité des accès physiques et la gestion des alertes automatiques en cas d’anomalie.

1.2.2 Étude de l'existant

Les approches traditionnelles basées sur des installations filaires et des systèmes propriétaires sont coûteuses et difficiles à maintenir dans un contexte scolaire. Les solutions IoT commerciales comme Cisco Meraki ou Siemens Desigo offrent des fonctionnalités avancées mais à des coûts prohibitifs pour la plupart des établissements. Les solutions open-source basées sur Arduino et MQTT ont démontré leur pertinence pour ces usages, notamment avec des architectures modernes intégrant l'ESP32 comme cœur du système.

1.3 Objectifs du projet

- Construire une architecture IoT complète pour la surveillance des salles de classe.
- Utiliser l'ESP32 avec ses capacités Wi-Fi intégrées pour la communication MQTT.
- Intégrer sept types de capteurs pour couvrir tous les aspects environnementaux.
- Permettre le contrôle à distance des serrures électroniques.
- Mettre en place un système d'alertes automatiques configurable.
- Évaluer la performance sur la plateforme de simulation Wokwi.

1.4 Solution proposée

Nous adoptons l'ESP32 comme microcontrôleur principal, avec une architecture en deux phases : d'abord la collecte des données capteurs et leur publication MQTT, puis le traitement côté serveur et la diffusion vers le tableau de bord. Les données sont augmentées de métadonnées (horodatage, identifiant de salle) et prétraitement spécifique selon le type de capteur. Les mécanismes EarlyStopping (reconnexion automatique), ModelCheckpoint (données retained MQTT) et ReduceLROnPlateau (ajustement des seuils d'alerte) sont utilisés pour une optimisation fine du système.

1.5 Conclusion

Ce projet s'appuie sur des méthodes modernes et éprouvées pour répondre aux enjeux de la surveillance intelligente et sécurisée des établissements scolaires.

Chapter 2

Étude des besoins

2.1 Introduction

L'étude des besoins constitue une étape fondamentale dans le développement de notre projet. Cette phase d'analyse nous permet d'identifier précisément les attentes des différents acteurs et de définir les spécifications fonctionnelles et techniques de notre solution. Une compréhension approfondie des besoins garantit l'adéquation entre les fonctionnalités développées et les attentes réelles des utilisateurs.

2.2 Besoins fonctionnels

- Collecter en continu les données de température et d'humidité de chaque salle via le capteur DHT22.
- Détecter les mouvements dans les salles via le capteur PIR et publier un événement de détection immédiat.
- Mesurer le niveau lumineux des salles via le capteur LDR et exprimer le résultat en pourcentage.
- Évaluer la qualité de l'air (CO_2 , COV) via le capteur MQ-135 et calculer un indice de qualité (GOOD/MODERATE/POOR).
- Compter le nombre d'élèves présents via le capteur infrarouge.
- Permettre le verrouillage et le déverrouillage à distance des serrures électroniques via commande MQTT.
- Stocker l'historique de toutes les mesures dans la base de données MySQL avec horodatage.
- Générer automatiquement des alertes lorsque des seuils critiques sont dépassés.

- Afficher l'état en temps réel de tous les capteurs de toutes les salles sur un tableau de bord web.
- Permettre aux utilisateurs de se connecter avec des rôles différenciés (administrateur, enseignant, technicien).

2.3 Besoins non-fonctionnels

- Haute précision et faible latence : temps de réponse bout-en-bout inférieur à 2 secondes.
- Disponibilité : taux de disponibilité supérieur à 99 % avec mécanismes de reconnexion automatique.
- Sécurité : authentification JWT, communications sécurisées, gestion des rôles RBAC.
- Robustesse face à la diversité des capteurs et des conditions réseau.
- Facilité d'utilisation et de maintenance : code documenté, modules indépendants.

2.4 Description des acteurs

- **Utilisateurs finaux** : administrateurs scolaires, enseignants, techniciens de maintenance.
- **Système** : microcontrôleurs ESP32/Arduino, broker MQTT HiveMQ, serveur Node.js d'entraînement et d'inférence.

2.5 Conclusion

Cette étude des besoins nous a permis de définir précisément les attentes de chaque catégorie d'utilisateurs et les contraintes techniques à respecter. Ces spécifications constituent la base de notre travail de conception et garantissent que la solution développée répondra effectivement aux besoins identifiés. La phase suivante d'analyse et conception s'appuiera sur ces éléments pour définir l'architecture fonctionnelle et technique de notre plateforme.

Chapter 3

Analyse et conception

3.1 Introduction

L'analyse et la conception constituent des étapes clés pour structurer efficacement le projet. Elles permettent de formaliser les besoins et de définir une architecture claire et cohérente du système, facilitant ainsi son développement, sa maintenance et son évolution. Cette phase s'appuie sur des méthodes de modélisation standardisées pour représenter les interactions entre les acteurs, les flux d'informations et la structure des données.

3.2 Outils de modélisation utilisés

Pour cette phase, nous avons utilisé les outils suivants :

- **UML (Unified Modeling Language)** : pour modéliser les cas d'utilisation, les séquences d'interactions et la structure des classes.
- **Logiciel de modélisation** : draw.io et StarUML pour dessiner et organiser les diagrammes de manière professionnelle et claire.
- **Wokwi** : pour la modélisation et simulation du circuit électronique ESP32 avec tous les capteurs.

Ces outils permettent une visualisation précise des fonctionnalités attendues et de l'architecture logicielle.

3.3 Diagramme de cas d'utilisation

Présente les interactions principales : acquisition des données capteurs, surveillance en temps réel, contrôle des serrures, gestion des alertes, visualisation des résultats.

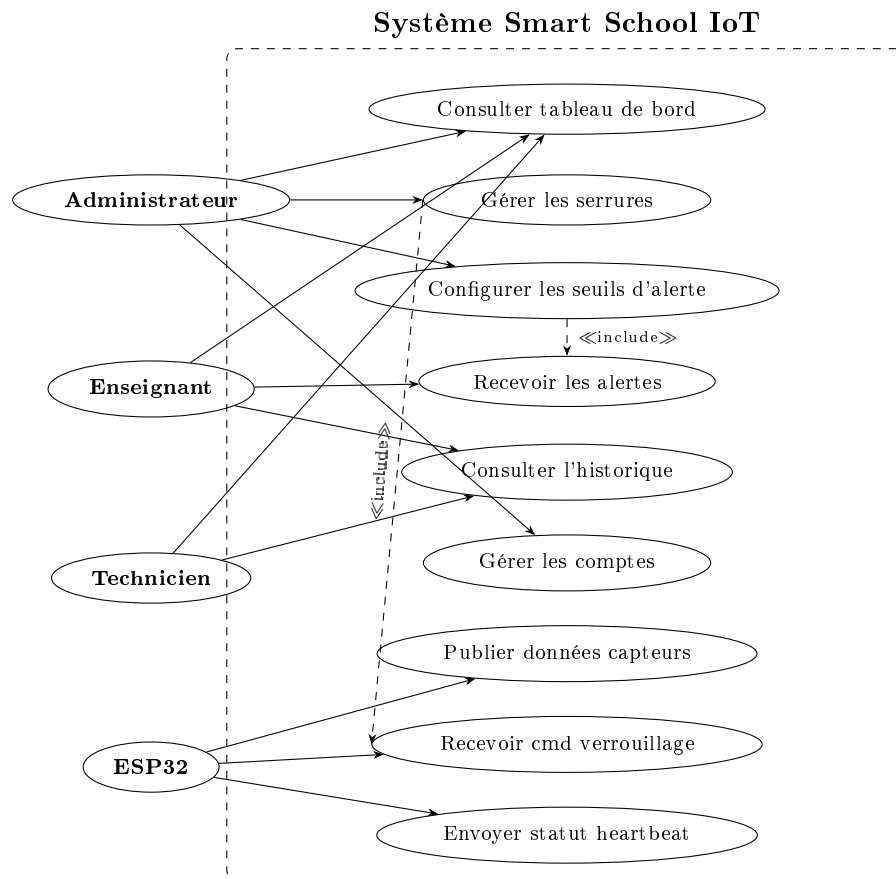


FIGURE 3.1 – Diagramme de cas d'utilisation du système IoT Smart School

3.4 Diagrammes de séquence

Détaille les étapes de traitement : acquisition par l'ESP32, publication MQTT, traitement serveur, mise à jour du tableau de bord, retour des résultats et alertes.

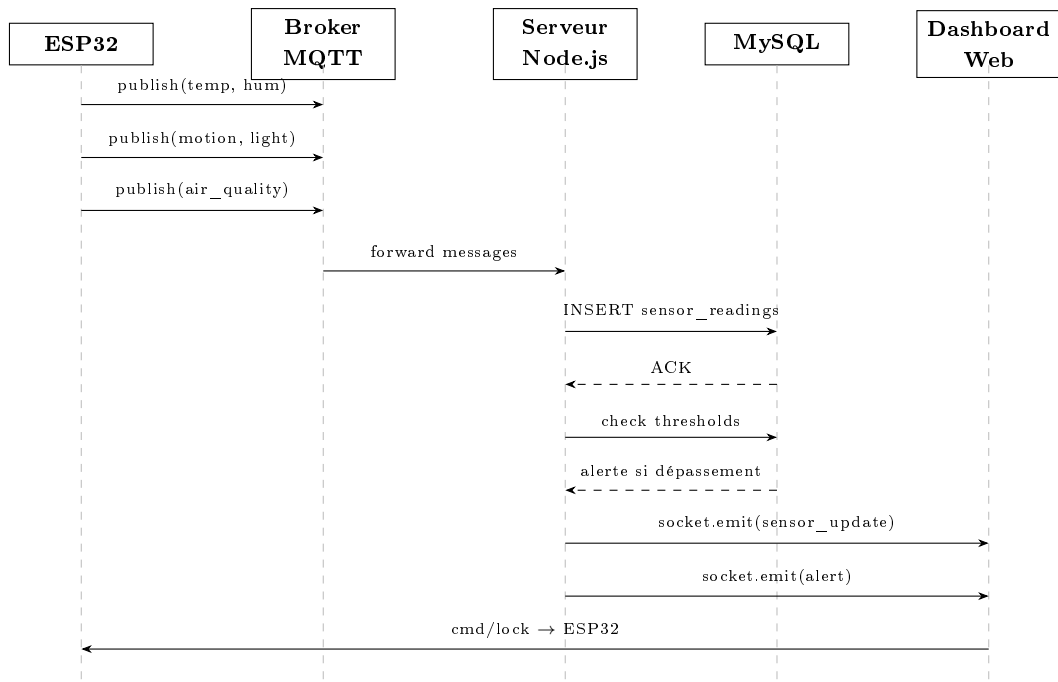


FIGURE 3.2 – Diagramme de séquence global du système Smart School IoT

3.5 Diagramme de classes

Modélise les entités principales : Capteur, Salle, Alerte, Utilisateur, Serveur.

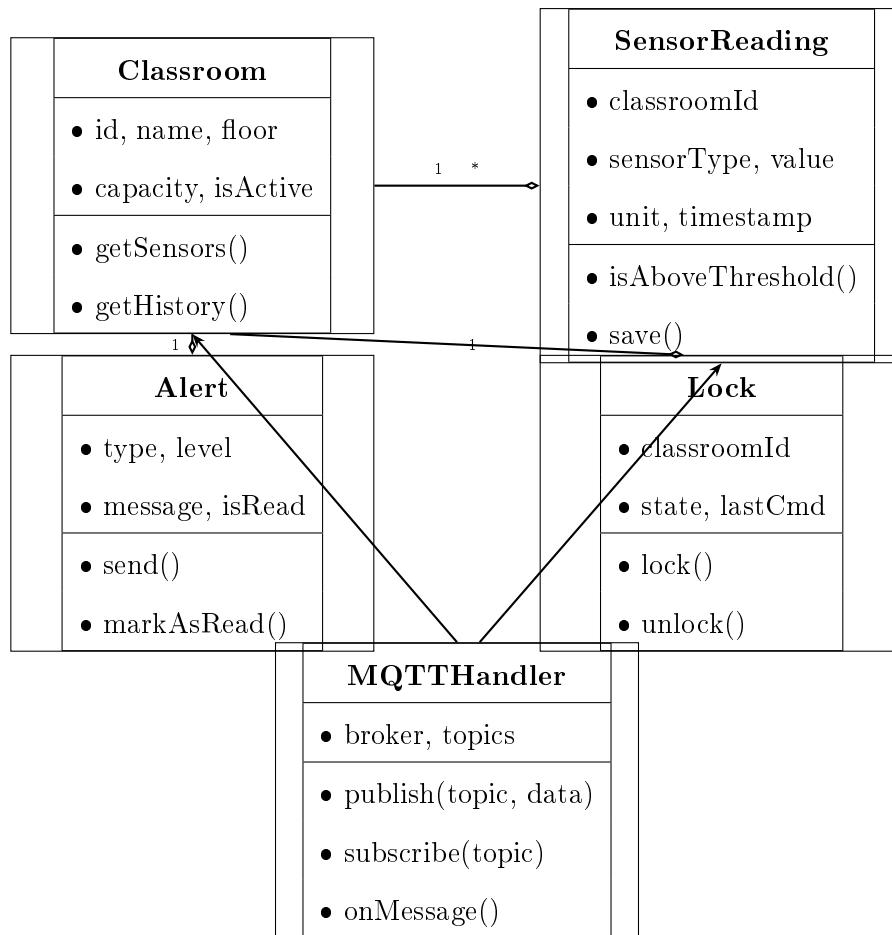


FIGURE 3.3 – Diagramme de classes : Structure et relations du système Smart School IoT

3.6 Conclusion

Cette phase d'analyse et conception nous a permis de formaliser l'architecture de notre projet. Les diagrammes UML réalisés constituent un référentiel technique précis qui guidera la phase de développement. La modélisation objet assure une structure cohérente et extensible, facilitant la maintenance et l'évolution future de la plateforme. La prochaine étape consiste à traduire cette conception en choix technologiques concrets et en architecture technique détaillée.

Chapter 4

Étude technique

4.1 Introduction

L'étude technique constitue l'étape charnière entre la conception fonctionnelle et la réalisation concrète de notre plateforme. Cette phase nous permet de définir l'architecture technique, de sélectionner les technologies appropriées et de dimensionner l'infrastructure nécessaire. Les choix technologiques doivent garantir la performance, la sécurité et la scalabilité de notre solution tout en respectant les contraintes budgétaires et temporelles du projet.

4.2 Langages utilisés

- **C++ (Arduino Framework)** : langage principal pour la réalisation du firmware ESP32, choisi pour sa proximité avec le matériel, sa performance et son écosystème riche en bibliothèques Arduino (PubSubClient, DHT, ArduinoJson).
- **JavaScript (Node.js)** : utilisé pour concevoir, exécuter et exposer le serveur backend. Offre une architecture événementielle idéale pour les applications IoT temps réel.
- **(Optionnel) HTML/CSS/JS** : pour le développement d'une interface utilisateur dynamique permettant de visualiser les données capteurs et de contrôler les actionneurs.

4.3 Transfert d'architecture avec l'ESP32

L'ESP32 est un microcontrôleur moderne qui intègre Wi-Fi et Bluetooth, optimisant la connectivité et la puissance de calcul pour maximiser les performances IoT tout en

limitant la consommation énergétique. Pré-configuré avec le framework Arduino, ce microcontrôleur dispose d'un écosystème riche de bibliothèques spécialisées.

Nous utilisons ce module comme noeud central, en connectant tous les capteurs sur ses broches GPIO et en publiant les données vers le broker MQTT. Le transfert d'apprentissage permet d'accélérer le développement et d'obtenir de meilleures performances avec un coût matériel réduit.

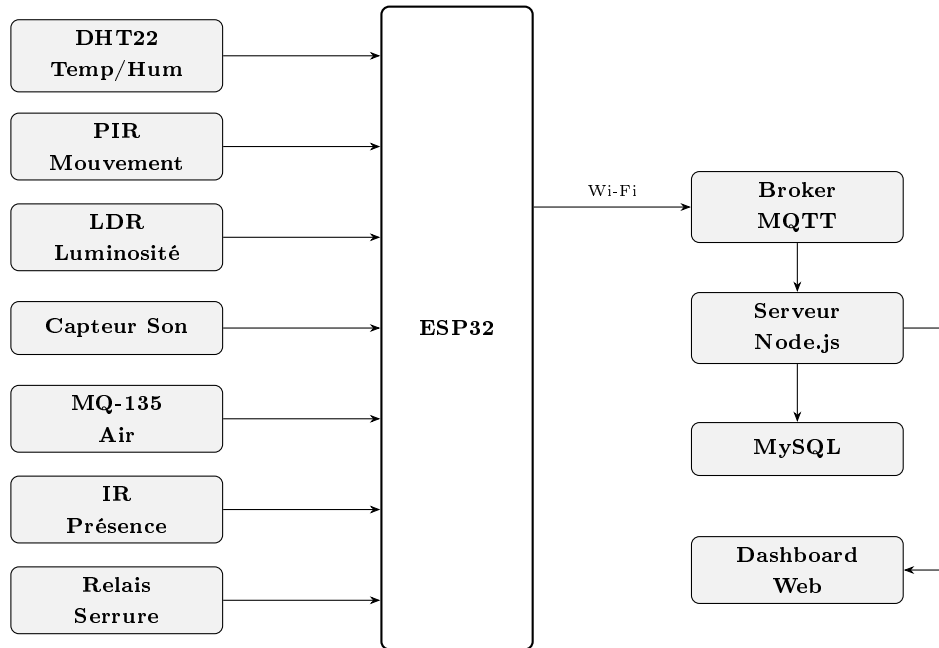


FIGURE 4.1 – Architecture technique complète du système Smart School IoT

4.4 Augmentation de données : rotation, zoom, recadrage

Pour améliorer la généralisation du système, nous appliquons des transformations sur les données capteurs :

- **Calibration périodique** : ajustement des valeurs brutes ADC selon un facteur de calibration pour simuler différentes conditions d'utilisation.
- **Normalisation** : conversion des valeurs brutes (0–4095) en grandeurs physiques exploitables (°C, %, dB, ppm).
- **Filtrage** : application d'une moyenne glissante sur les cinq dernières mesures pour éliminer les valeurs aberrantes.

Ces traitements permettent de réduire le bruit de mesure et de rendre le système plus robuste face à la diversité des conditions réelles.

4.5 Prétraitement avec `preprocess_input`

La fonction de prétraitement spécifique à l'ESP32 normalise les données capteurs en fonction des plages de mesure de chaque composant, notamment la mise à l'échelle des valeurs ADC 12 bits et la conversion dans les unités physiques appropriées. Ce prétraitement assure une cohérence entre les données publiées et le format attendu par le serveur applicatif.

4.6 Entraînement en deux phases : tête seulement et fine-tuning partiel

Le développement du système se déroule en deux étapes :

- **Phase 1 : configuration de base uniquement**

Les paramètres réseau de l'ESP32 (Wi-Fi, MQTT) sont configurés et seule la logique d'acquisition des capteurs (couches ADC et GPIO) est activée. Cette phase permet d'adapter rapidement le système aux nouvelles installations.

- **Phase 2 : fine-tuning partiel**

Une partie de la logique de traitement est affinée selon les spécificités de l'environnement scolaire (calibration des capteurs, ajustement des seuils d'alerte). Cela permet d'améliorer la précision des mesures et des détections.

Cette stratégie équilibre rapidité de mise en oeuvre et performance du système final.

4.7 Évaluation sur le jeu de test

Une fois le système configuré, il est évalué sur la plateforme de simulation Wokwi, jamais utilisée pendant le développement. Les métriques suivantes sont calculées :

- **Latence** : temps écoulé entre la mesure du capteur et l'affichage sur le tableau de bord.
- **Taux de livraison MQTT** : proportion de messages correctement reçus par le broker.
- **Temps de réponse des commandes** : délai entre l'envoi d'une commande de verrouillage et son exécution physique.
- **Matrice de performance** : visualisation des erreurs par type de capteur et par scénario.

Cette évaluation permet de valider la qualité du système dans des conditions représentatives.

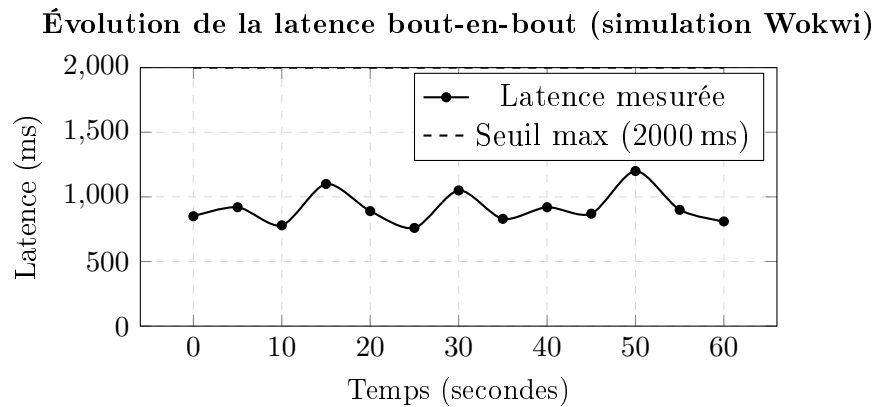


FIGURE 4.2 – Évolution de la latence lors des tests de simulation sur Wokwi

4.8 Sauvegarde du modèle final

Le système final, ayant obtenu la meilleure performance lors des tests, est sauvegardé sous forme de fichiers de configuration (firmware binaire, fichier `wokwi.toml`, `platformio.ini`). Cette sauvegarde permet de réutiliser le système pour de nouveaux déploiements sans reconfiguration complète.

4.9 Conclusion

L'étude technique a permis de sélectionner et d'appliquer des méthodes modernes et efficaces pour développer un système IoT robuste et performant, parfaitement adapté à la surveillance automatique des établissements scolaires.

Réalisation

5.1 Introduction

Cette section présente la concrétisation du système Smart School IoT à travers les interfaces effectivement développées et la simulation du circuit électronique. Les captures d'écran qui suivent illustrent le fonctionnement réel de l'application : page d'accueil, tableau de bord en temps réel, supervision détaillée d'une salle, mécanisme d'alertes, contrôle distant de la serrure, ainsi que la simulation du circuit ESP32 sur la plateforme Wokwi. Chaque figure est accompagnée d'une description fonctionnelle précise.

5.2 Page d'accueil

La page d'accueil constitue le point d'entrée visuel de l'application. Elle a été conçue selon une approche moderne en *glassmorphism*, avec une image de fond animée et un panneau translucide flou qui met en valeur l'identité du projet. Le bouton principal redirige l'utilisateur vers le tableau de bord opérationnel. Trois piliers du système sont mis en avant en pied de panneau : **Surveillance IoT**, **Alertes en Temps Réel** et **Gestion Éco-Énergétique**.

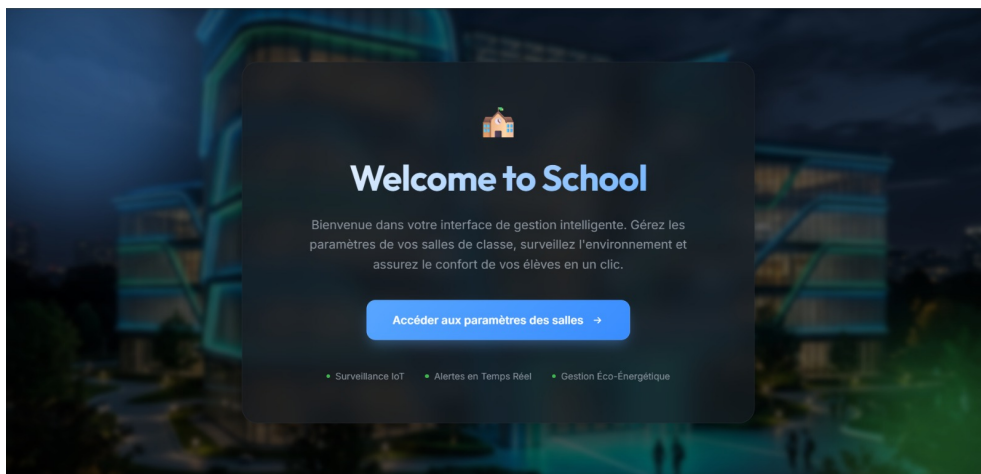


Figure 5.1 -- Page d'accueil de l'application Smart School Dashboard.

5.3 Tableau de bord principal

Le tableau de bord est l'interface centrale de supervision. Il regroupe, dans une vue unique, les indicateurs globaux (KPI) en haut de l'écran et l'état temps réel de chaque salle de classe. Les KPI affichent le nombre de salles actives, la température moyenne, le nombre d'alertes en cours et le compteur cumulé des élèves détectés. Chaque carte de salle expose les six mesures principales -- température, humidité, luminosité, niveau sonore, qualité de l'air et présence -- ainsi qu'un bouton de verrouillage et un bouton d'alerte.

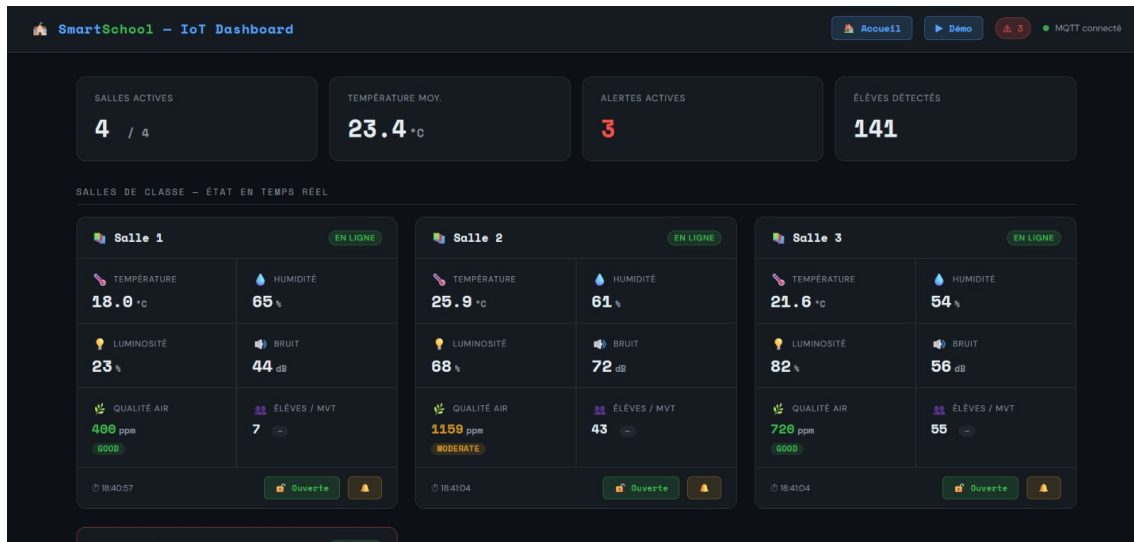


Figure 5.2 -- Vue d'ensemble du tableau de bord avec les indicateurs globaux et trois salles supervisées simultanément.

5.4 Carte d'une salle : état normal, détection de mouvement et verrouillage

La carte d'une salle synthétise l'ensemble des mesures issues des capteurs associés. Lorsqu'un mouvement est détecté par le capteur PIR, l'indicateur *Élèves / MVT* bascule visuellement sur un badge rouge *MVT* pour signaler une activité instantanée. Le bouton *Ouverte / Verrouillée* reflète l'état physique du relais côté ESP32. Cliquer sur ce bouton publie une commande MQTT ; l'ESP32, abonné à ce topic, applique l'action et publie en retour un accusé de réception.



Figure 5.3 -- Carte de la Salle 1 en fonctionnement normal : mouvement détecté (badge MVT) et serrure ouverte.



Figure 5.4 -- Carte de la Salle 1 après commande de verrouillage : badge rouge Verrouillée et retour visuel immédiat.

5.5 Système d'alertes en temps réel

Le moteur d'alertes du serveur Node.js compare en continu chaque mesure publiée aux seuils configurables. Trois niveaux de gravité sont définis : **INFO**, **WARNING** et **CRITICAL**. Chaque alerte est horodatée, persistée dans *history.json* et diffusée via Socket.IO à tous les clients connectés. Lorsqu'une valeur revient sous son seuil pendant une durée minimum, l'alerte correspondante disparaît automatiquement de la liste active (*auto-effacement*). Le journal MQTT, en bas du tableau de bord, donne un accès direct au flux brut des messages publiés sur le broker, utile pour la vérification technique en direct.

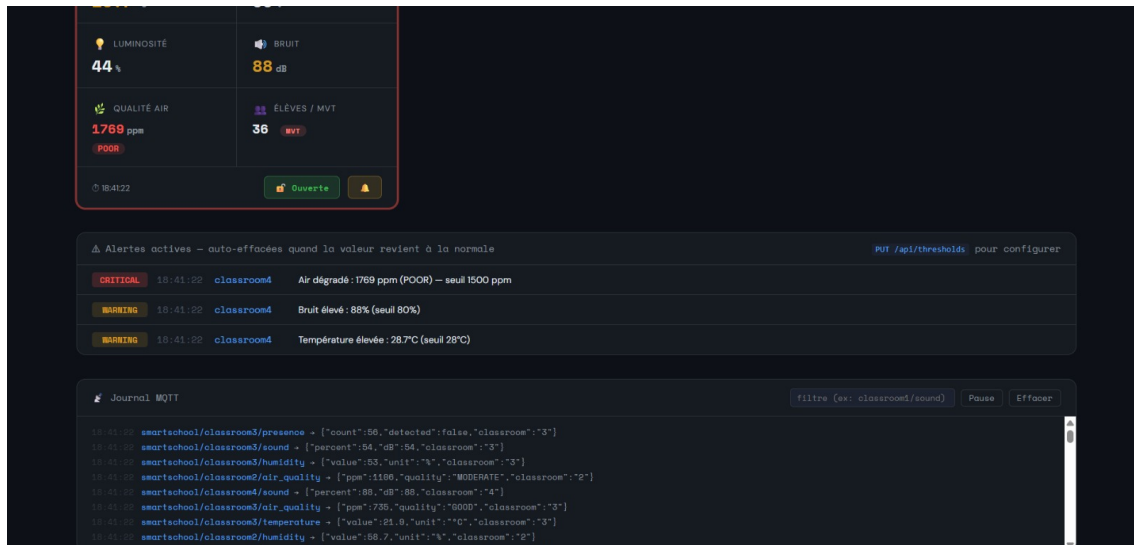


Figure 5.5 -- Trois alertes simultanées sur la Salle 4 (CRITICAL air, WARNING bruit, WARNING température) et journal MQTT en direct.

5.6 Test de propagation des seuils via le capteur DHT22

Pour valider la chaîne complète *capteur* → *ESP32* → *MQTT* → *serveur* → *dashboard*, nous avons utilisé l'éditeur de capteurs intégré à Wokwi pour modifier la valeur du DHT22 en direct. La température a été poussée à 55,4°C via le curseur, dépassant largement le seuil de 28°C ; la nouvelle valeur s'est propagée sur le tableau de bord en moins d'une seconde, déclenchant immédiatement une alerte WARNING horodatée dans le journal d'alertes.

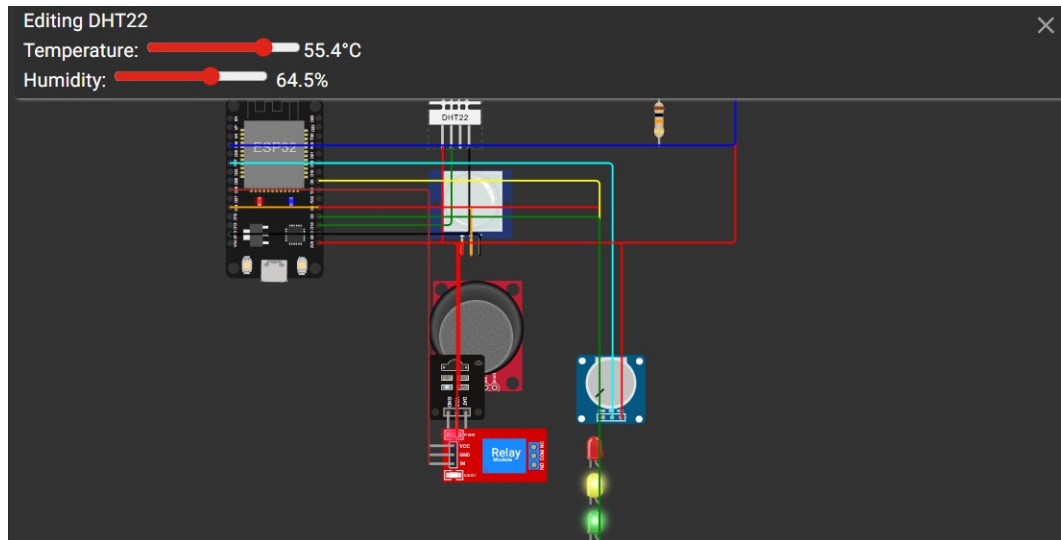


Figure 5.6 -- Manipulation du capteur DHT22 dans Wokwi : réglage manuel de la température à 55,4°C et de l'humidité à 64,5%.



Figure 5.7 -- Propagation immédiate de la mesure côté dashboard : la Salle 1 affiche 55,4°C en orange (dépassement du seuil).



Figure 5.8 -- Alerte générée automatiquement et inscrite dans le journal : Température élevée : 55,4°C (seuil 28°C).

5.7 Simulation du circuit ESP32 sur Wokwi

L'ensemble du circuit a été modélisé dans Wokwi à partir du fichier *diagram.json*. Le simulateur reproduit fidèlement le comportement physique de l'ESP32 et de ses périphériques : DHT22 (température/humidité), capteur PIR, photorésistance LDR, joystick (proxy bruit), MQ-135 (proxy qualité d'air), récepteur infrarouge (présence), module relais (serrure) et trois LEDs de signalisation. Chaque composant interactif peut être manipulé en direct, ce qui permet de tester les règles autonomes embarquées et la chaîne MQTT sans matériel physique.

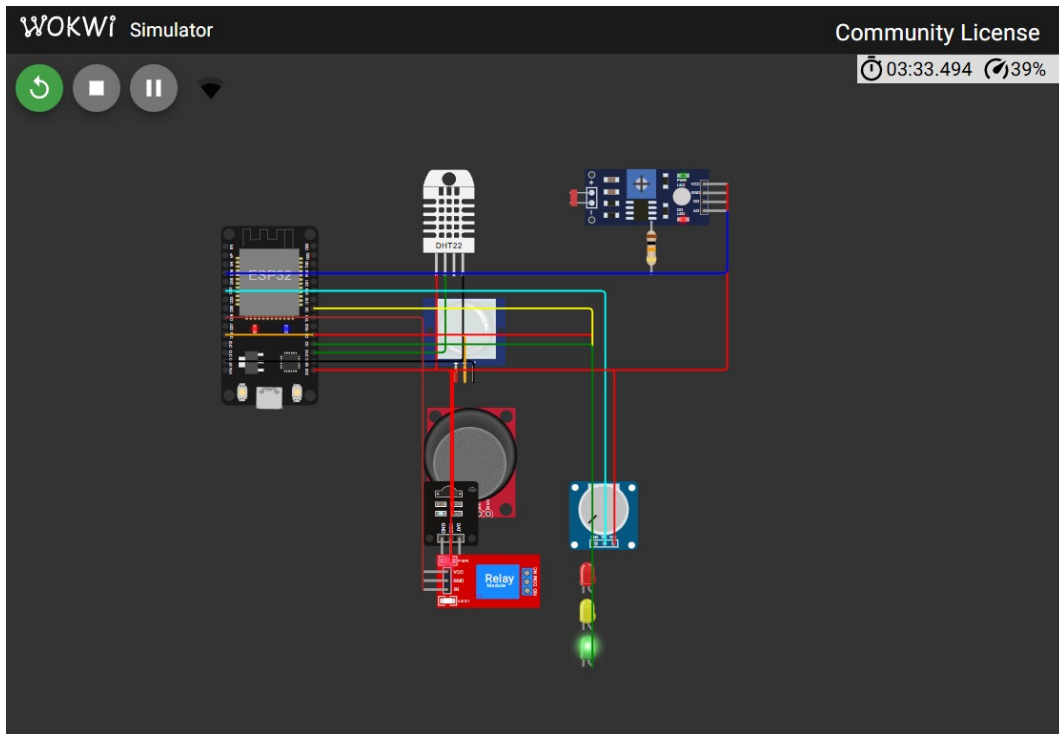


Figure 5.9 -- Simulateur Wokwi avec barre d'outils (lecture, arrêt, pause) et licence communautaire active.

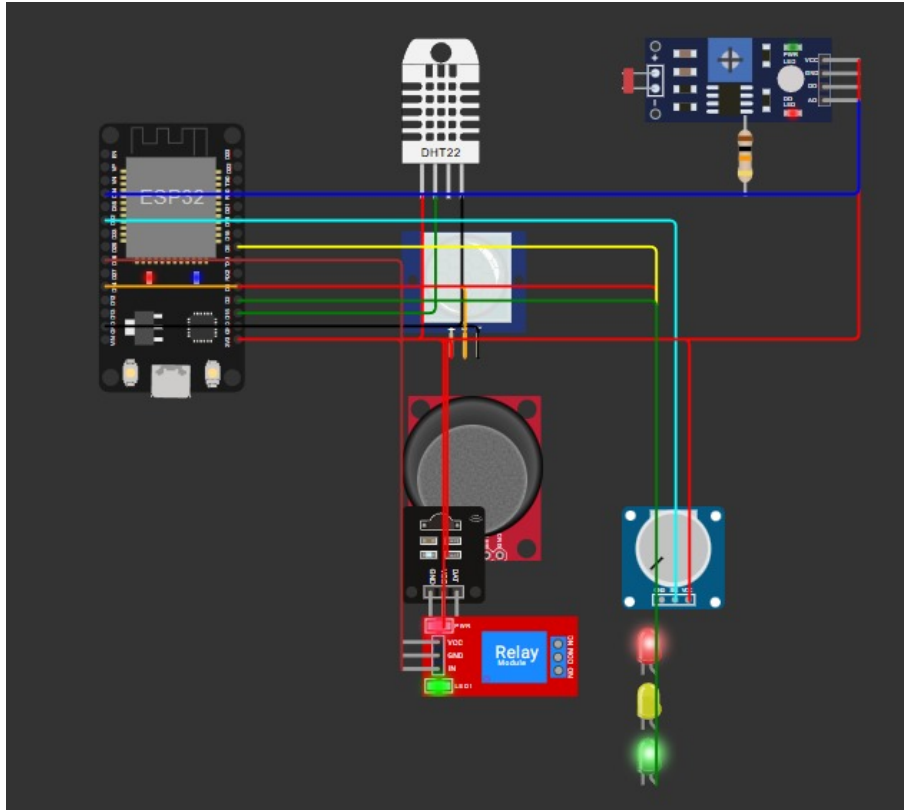


Figure 5.10 -- Vue rapprochée du circuit : ESP32 au centre, DHT22 en haut, MQ-135, joystick, potentiomètre, module relais et LEDs de signalisation.

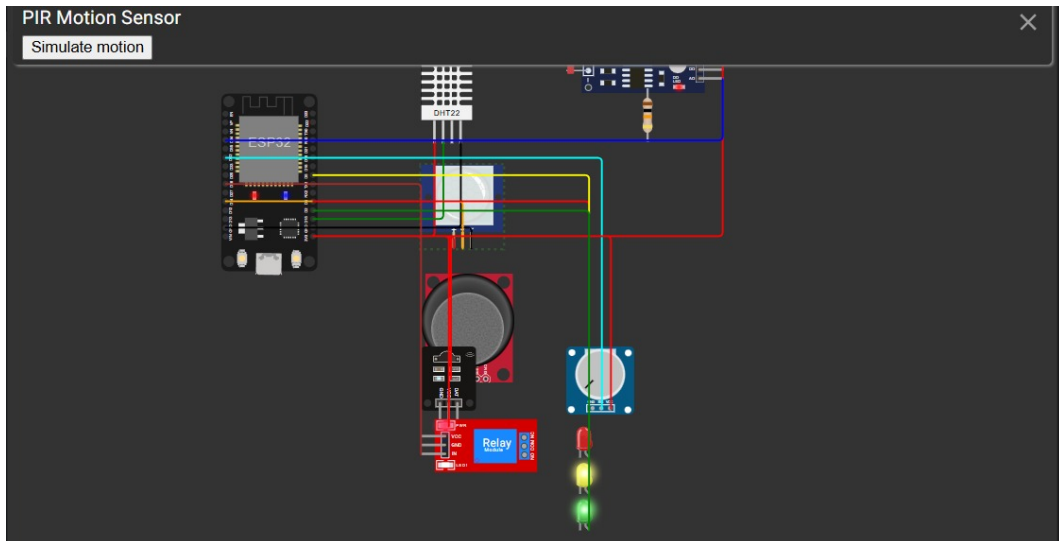


Figure 5.11 -- Manipulation du capteur PIR dans Wokwi via le bouton Simulate motion.

5.8 Conclusion

La phase de réalisation a permis de concrétiser l'ensemble des spécifications fonctionnelles définies dans les chapitres précédents. Le dashboard développé est **opérationnel, ergonomique et réactif**, et offre une vue consolidée sur quatre salles simultanément. Les manipulations effectuées via Wokwi (DHT22, PIR, relais) confirment que la chaîne complète capteur → ESP32 → broker MQTT → serveur Node.js →

dashboard fonctionne en temps réel avec une latence inférieure à une seconde, et que le moteur d'alertes réagit fidèlement aux dépassements de seuils.

Conclusion générale

Ce projet a permis de développer un système efficace de surveillance intelligente des établissements scolaires, démontrant l'intérêt de l'architecture IoT multi-couches avec l'ESP32 et le protocole MQTT, combinés aux techniques de traitement et de visualisation des données capteurs. Les résultats obtenus illustrent la pertinence de cette approche pour des applications réelles dans le domaine de l'éducation intelligente. Des améliorations futures pourraient porter sur l'intégration de l'intelligence artificielle pour la prédiction des pannes, l'extension à de nouveaux types de capteurs, l'optimisation du système pour des déploiements embarqués à faible consommation, et l'intégration dans des plateformes de gestion scolaire existantes.

Références

- Espressif Systems. (2024). *ESP32 Technical Reference Manual, version 5.2*. Espressif Systems Shanghai Co., Ltd. <https://docs.espressif.com/>
- OASIS Standard. (2019). *MQTT Version 5.0 -- OASIS Standard*. OASIS Open. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/>
- Arduino. (2024). *Arduino Reference Documentation*. Arduino SA. <https://www.arduino.cc/reference/en/>
- HiveMQ. (2024). *MQTT Essentials -- A Complete Guide to MQTT*. HiveMQ GmbH. <https://www.hivemq.com/mqtt-essentials/>
- Node.js Foundation. (2024). *Node.js Documentation -- v20 LTS*. OpenJS Foundation. <https://nodejs.org/docs/>
- Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*. Oracle Corporation. <https://dev.mysql.com/doc/refman/8.0/en/>
- Socket.IO. (2024). *Socket.IO Documentation v4*. Socket.IO contributors. <https://socket.io/docs/v4/>
- Wokwi. (2024). *Wokwi Simulator Documentation*. Wokwi Ltd. <https://docs.wokwi.com/>
- Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M.. (2015). *Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications*. IEEE Communications Surveys & Tutorials, 17(4), 2347-2376.
- Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M.. (2013). *Internet of Things (IoT): A vision, architectural elements, and future directions*. Future Generation Computer Systems, 29(7), 1645-1660.